

深圳大学实验报告

课程名称 Python程序设计
项目名称 简易图书管理系统
学 院 人工智能学院
专 业 计算机科学技术
指导教师 舒婷、樊超
报 告 人 朱明钊 学号 2024341011
实验时间 2025年11月9日
提交时间 2025年11月9日

教务处

目录

二、 基础功能要求（90 分）	2
1. 数据结构设计（10 分）	2
2. 核心功能实现（共 80 分）	3
三、 附加功能（10 分）	3
四、 实验环境	4
五、 功能实现与代码展示	4
1. 定义一个 Book 类，包含图书的核心属性：图书 ID（唯一标识）、书名、作	4
2. 封装一个Library类	5
六、 运行结果展示	8
七、 实验总结与心得：	11
1. 通过完成"简易图书馆资源管理系统"实验，我在以下方面收获颇丰：	11
2. 遇到的问题及解决方法	11
3. 对OOP的理解提升	12
4. 心得体会	12

实验目标：

- 1、熟练运用 Python 的核心数据结构（列表、字典）进行数据存储与处理。
- 2、掌握函数的定义与调用，理解代码模块化思想。
- 3、理解面向对象编程（OOP）思想，能够设计并使用类封装数据与功能。
- 4、培养问题分析、系统设计及代码实现能力。

二、基础功能要求（90 分）

设计一个“简易图书馆资源管理系统”，需综合运用列表、字典、函数和类，实现以下功能：

1. 数据结构设计（10 分）

定义 Book 类，包含属性：

✧ 定义一个 Book 类，包含图书的核心属性：图书 ID（唯一标识）、书名、作

者、出版社、是否可借（布尔值）。

✧ 使用列表存储系统中所有图书（Book 类的实例）。

✧ 使用字典存储借阅记录，键为“图书 ID”，值为“借阅人姓名”。

2. 核心功能实现（共 80 分）

封装一个 Library 类，实现以下方法：

✧添加图书：接收用户输入的图书信息（ID、书名、作者等），创建 Book 实

例并添加到图书列表（需检查 ID 唯一性，避免重复）。

✧查询图书：支持两种查询方式（通过函数参数区分）：

■按“书名”模糊查询（返回所有包含关键词的图书列表）；

■按“作者”精确查询（返回该作者的所有图书列表）。

✧修改图书信息：根据图书 ID 查找图书，允许修改除 ID 外的其他属性（如更新出版社）。

✧删除图书：根据图书 ID 删除图书（需先检查是否已被借阅，若已借阅则不允许删除）。

✧借阅图书：接收图书 ID 和借阅人姓名，修改图书的“可借状态”为 False，并记录到借阅字典中（需检查图书是否可借）。

✧归还图书：根据图书 ID，修改图书的“可借状态”为 True，并从借阅字典中移除记录。

三、附加功能（10 分）

从以下方向选择 1-2 项实现，或自主设计其他合理功能：

✧数据持久化：将图书列表和借阅记录保存到本地文件（如txt、json），程序启动时自动加载。

✧高级查询：支持按“出版社”“是否可借”等条件组合查询，或按“图书 ID”/“书名”排序后展示。

✧借阅统计：新增属性记录每本书的借阅次数，或统计指定借阅人的历史借阅记录。

✧用户权限：区分“管理员”和“普通用户”角色（管理员可执行所有作，普通用户仅能查询和借阅）。

✧图形界面：使用 tkinter 库设计简单的可视化操作界面（如按钮、输入框）。

四、实验环境

Windows 11中文版

IDE: Visual Studio2026 Insider

解释器: Python 3.12.8

五、功能实现与代码展示

1. 定义一个 Book 类，包含图书的核心属性：图书 ID（唯一标识）、书名、作者、出版社、是否可借（布尔值）

```
class Book:
    """
    定义一个 Book 类，包含图书的核心属性：图书 ID（唯一标识）、书名、作者、出版社、是否可借（布尔值）。
    """

    #默认构造函数
    def __init__(self, book_id, title, author, publisher, is_available=True, borrow_count=0):
        self.book_id=str(book_id)
        self.title=title
        self.author=author
        self.publisher=publisher
        self.is_available=bool(is_available)
        self.borrow_count = int(borrow_count)

    #打印图书信息
    def print_book_info(self):
        print(f"图书ID: {self.book_id}")
        print(f"书名: 《{self.title}》")
        print(f"作者: {self.author}")
        print(f"出版社: {self.publisher}")
        print(f"是否可借: {self.is_available}")
        print(f"借阅次数: {self.borrow_count}")

    def to_dict(self):
        return{
            "book_id":self.book_id,
            "title":self.title,
            "author":self.author,
            "publisher":self.publisher,
            "is_available":self.is_available,
            "borrow_count":self.borrow_count,
        }
```

```

97     @staticmethod
98     def from_dict(data:dict):
99         return Book(
100             book_id=data["book_id"],
101             title=data["title"],
102             author=data["author"],
103             publisher=data["publisher"],
104             is_available=data.get("is_available",True),
105             borrow_count=data.get("borrow_count",0),
106         )
107

```

2. 封装一个 Library 类

```

126 class Library:
127     """
128     封装一个 Library 类
129     - self.books:列表, 存储Book实例
130     - self.borrow_records:字典, 键为图书ID, 值为借阅人姓名
131     """
132
133     def __init__(self, data_file:str|None=None):
134         # 用于存储所有图书(Book实例)的列表
135         self.books=[]
136         # 用于存储借阅记录, 键为图书ID, 值为借阅人 user_name
137         self.borrow_records={}
138         self.data_file=data_file
139         if self.data_file and os.path.exists(self.data_file):
140             self.load_from_file(self.data_file)
141
142     #工具方法: 根据ID查找图书
143     def find_book_by_id(self, book_id):
144         for book in self.books:
145             if book.book_id == str(book_id):
146                 return book
147         return None
148

```

1) 添加图书：接收用户输入的图书信息（ID、书名、作者等），创建 Book 实例并添加到图书列表（需检查 ID 唯一性，避免重复）。

```

149     def add_book(self, book_id, title, author, publisher, is_available=True):
150         """
151         添加图书: 接收用户输入的图书信息 (ID、书名、作者等),
152         创建 Book 实例并添加到图书列表 (需检查 ID 唯一性, 避免重复)
153         """
154         # 检查ID唯一性
155         if self.find_book_by_id(book_id) is not None:
156             print(f"图书ID {book_id} 已存在, 不能重复添加!")
157             return False
158
159         #创建Book实例并添加
160         new_book = Book(book_id, title, author, publisher, is_available)
161         self.books.append(new_book)
162         print(f"图书《{title}》已成功添加!")
163         return True
164

```

2) 查询图书：支持两种查询方式（通过函数参数区分）：

- 按 “书名” 模糊查询（返回所有包含关键词的图书列表）；
- 按 “作者” 精确查询（返回该作者的所有图书列表）。

```

166 def search_book(self, value, mode="title"):
167     """
168     综合查询方法
169     :param value: 查询关键词或作者名
170     :param mode: 查询模式, "title"为书名模糊查询, "author"为作者精确查询
171     :return: 匹配的图书列表
172     """
173     result = []
174     if mode == "title":
175         keyword = str(value).lower()
176         for book in self.books:
177             if keyword in book.title:
178                 result.append(book)
179     elif mode == "author":
180         for book in self.books:
181             if str(value) == book.author:
182                 result.append(book)
183     else:
184         print("查询模式不正确, 应为'title' 或 'author'")
185     return result
186
187 #高级查询功能
188 def advanced_query(self, publisher=None, is_available=None, sort_by=None):
189     result = self.books
190     if publisher is not None:
191         result = [b for b in result if b.publisher == publisher]
192     if is_available is not None:
193         result = [b for b in result if b.is_available == is_available]
194     if sort_by == "book_id":
195         result = sorted(result, key=lambda b: b.book_id)
196     elif sort_by == "title":
197         result = sorted(result, key=lambda b: b.title)
198     elif sort_by == "borrow_count":
199         result = sorted(result, key=lambda b: b.borrow_count, reverse=True)
200     return result
201

```

3) 修改图书信息：根据图书 ID 查找图书，允许修改除 ID 外的其他属性（如更新出版社）。

```

202 def modify_book(self, book_id, title=None, author=None, publisher=None, is_available=None):
203     """
204     修改图书信息：根据图书 ID 查找图书，允许修改除 ID 外的其他属性
205     : param book_id: 要修改的图书ID
206     : param title: 新书名（可选）
207     : param author: 新作者（可选）
208     : param publisher: 新出版社（可选）
209     : param is_available: 新可借情况（可选）
210     """
211     for book in self.books:
212         if book.book_id == book_id:
213             if title is not None:
214                 book.title = title
215             if author is not None:
216                 book.author = author
217             if publisher is not None:
218                 book.publisher = publisher
219             if is_available is not None:
220                 book.is_available = is_available
221             print(f"图书ID{book_id} 信息已更新")
222             book.print_book_info()
223             return True
224     print(f"未找到图书ID {book_id}，无法更新图书信息")
225     return False
226

```

4) 删除图书：根据图书 ID 删除图书（需先检查是否已被借阅，若已借阅则不允许删除）。

```

227 def remove_book(self, book_id):
228     """
229     删除图书：根据图书 ID 删除图书
230     (需先检查是否已被借阅，若已借阅则不允许删除)
231     """
232     for book in self.books:
233         if book.book_id == book_id:
234             # 检查借阅记录
235             if book_id in self.borrow_records:
236                 print(f"图书 {book.title} 正被借阅，无法删除")
237                 return False
238             # 如果未被借阅，则可以删除
239             self.books.remove(book)
240             print(f"图书 {book.title} 已被删除")
241             return True
242     print(f"未找到图书ID {book_id}")
243     return False

```

5) 借阅图书：接收图书 ID 和借阅人姓名，修改图书的“可借状态”为 False，并记录到借阅字典中（需检查图书是否可借）。

```
245 def borrow_book(self, book_id, user_name):
246     """
247     借阅图书：接收图书 ID 和借阅人姓名，修改图书的“可借状态”为 False，
248     并记录到借阅字典中（需检查图书是否可借）。
249     """
250     for book in self.books:
251         if book.book_id == book_id:
252             #检查借阅记录
253             if book_id in self.borrow_records:
254                 print(f"图书《{book.title}》已被借阅")
255                 return False
256             #如果可以此图书可被借阅
257             book.is_available = False
258             self.borrow_records[book_id] = user_name
259             book.borrow_count += 1
260             print(f"图书《{book.title}》已借给{user_name}")
261
```

6) 归还图书：根据图书 ID，修改图书的“可借状态”为 True，并从借阅字典中移除记录。

```
263 def return_book(self, book_id):
264     """
265     归还图书：根据图书 ID，修改图书的“可借状态”为 True，
266     并从借阅字典中移除记录。
267     """
268     for book in self.books:
269         #检查借阅记录
270         if book_id == book.book_id:
271             if book_id in self.borrow_records:
272                 book.is_available = True
273                 del self.borrow_records[book_id]
274                 print(f"图书《{book.title}》已归还")
275                 return True
276             else:
277                 print(f"图书《{book.title}》未被借出，无需归还")
278                 return False
279     print(f"未找到图书{book_id}，无法归还")
280     return False
281
```

附加功能实现：

1) 数据持久化：将图书列表和借阅记录保存到本地文件（如txt、json），程序启动时自动加载。

```
283 # 存储借阅数据
284 def save_to_file(self, path=None):
285     file_path = path or self.data_file
286     if not file_path:
287         print("未指定保存文件路径。")
288         return False
289     data = {
290         "books": [b.to_dict() for b in self.books],
291         "borrow_records": self.borrow_records,
292     }
293     with open(file_path, "w", encoding="utf-8") as f:
294         json.dump(data, f, ensure_ascii=False, indent=2)
295     print(f"数据已保存到{file_path}")
296     return True
297
298 #加载借阅数据
299 def load_from_file(self, path):
300     with open(path, "r", encoding="utf-8") as f:
301         data = json.load(f)
302     self.books = [Book.from_dict(d) for d in data.get("books", [])]
303     self.borrow_records = data.get("borrow_records", {})
304     print(f"已从{path}加载数据：图书{len(self.books)}本，借阅记录{len(self.borrow_records)}条")
305
306
```

2) 图形界面：使用 tkinter 库设计简单的可视化操作界面（如按钮、输入框）

```
376 class LibraryGUI:
377     def __init__(self, master, library):
378         self.master = master
379         self.library = library
380         master.title("简易图书馆管理系统")
381
382         # 添加图书
383         tk.Label(master, text="添加图书").grid(row=0, column=0, columnspan=2, pady=5)
384         tk.Label(master, text="ID").grid(row=1, column=0)
385         tk.Label(master, text="书名").grid(row=2, column=0)
386         tk.Label(master, text="作者").grid(row=3, column=0)
387         tk.Label(master, text="出版社").grid(row=4, column=0)
388         self.entry_id = tk.Entry(master)
389         self.entry_title = tk.Entry(master)
390         self.entry_author = tk.Entry(master)
391         self.entry_publisher = tk.Entry(master)
392         self.entry_id.grid(row=1, column=1)
393         self.entry_title.grid(row=2, column=1)
394         self.entry_author.grid(row=3, column=1)
395         self.entry_publisher.grid(row=4, column=1)
396         tk.Button(master, text="添加", command=self.add_book).grid(row=5, column=0, columnspan=2, pady=5)
397
398         # 查询图书
399         tk.Label(master, text="查询图书").grid(row=6, column=0, columnspan=2, pady=5)
400         self.entry_search = tk.Entry(master)
401         self.entry_search.grid(row=7, column=0)
402         tk.Button(master, text="按书名查询", command=self.search_by_title).grid(row=7, column=1)
403         tk.Button(master, text="按作者查询", command=self.search_by_author).grid(row=8, column=1)
404
405         # 借阅/归还
406         tk.Label(master, text="借阅/归还").grid(row=9, column=0, columnspan=2, pady=5)
407         self.entry_borrow_id = tk.Entry(master)
408         self.entry_borrow_id.grid(row=10, column=0)
409         tk.Button(master, text="借阅", command=self.borrow_book).grid(row=10, column=1)
410         tk.Button(master, text="归还", command=self.return_book).grid(row=11, column=1)
411
412         # 显示所有图书
413         tk.Button(master, text="显示所有图书", command=self.show_all_books).grid(row=12, column=0, columnspan=2, pady=5)
```

3) 借阅统计：新增属性记录每本书的借阅次数，或统计指定借阅人的历史借阅记录。

六、运行结果展示

C:\Users\zhu72\AppData\Loca + v

已从library_data.json加载数据：图书4本，借阅记录0条
图书ID001已存在，不能重复添加！
图书ID002已存在，不能重复添加！
图书ID003已存在，不能重复添加！
图书《简明Python教程》已成功添加！
图书ID005已存在，不能重复添加！

【1】当前图书馆藏书：
图书ID: 001
书名：《Python编程:从入门到实践》
作者：[美]埃里克·马瑟斯(Eric Matthes)
出版社：人民邮电出版社
是否可借：True
借阅次数：3

图书ID: 002
书名：《流畅的Python》
作者：Luciano Ramalho
出版社：人民邮电出版社
是否可借：True
借阅次数：0

图书ID: 003
书名：《Python核心编程》
作者：Wesley J. Chun
出版社：人民邮电出版社
是否可借：True
借阅次数：0

图书ID: 005
书名：《Python数据科学手册》
作者：Jake VanderPlas
出版社：人民邮电出版社
是否可借：True
借阅次数：0

图书ID: 004
书名：《简明Python教程》
作者：Swaroop C H
出版社：机械工业出版社
是否可借：True
借阅次数：0

【2】按书名模糊查询（关键词 'Python'）：

简易图书馆管理系统

添加图书

ID

书名

作者

出版社

添加

查询图书

按书名查询

按作者查询

借阅/归还

借阅

归还

显示所有图书

```
C:\Users\zhu72\AppData\Loca  ×  +  v

-----

【2】按书名模糊查询（关键词 'Python'）：

【3】按作者精确查询（作者 'Luciano Ramalho'）：
图书ID: 002
书名：《流畅的Python》
作者：Luciano Ramalho
出版社：人民邮电出版社
是否可借：True
借阅次数：0
-----

【4】修改图书信息（ID '003'，出版社改为 '人民邮电出版社'）：
图书ID003信息已更新
图书ID: 003
书名：《Python核心编程》
作者：Wesley J. Chun
出版社：人民邮电出版社
是否可借：True
借阅次数：0

【5】删除图书（ID '004'）：
图书简明Python教程已被删除

【6】借阅图书（ID '001'，借阅人 '王五'）：
图书《Python编程:从入门到实践》已借给王五

【7】再次尝试借阅同一本书（ID '001'）：
图书《Python编程:从入门到实践》已被借阅

【8】归还图书（ID '001'）：
图书《Python编程:从入门到实践》已归还

【9】高级查询（出版社 '人民邮电出版社'，按借阅次数降序）：
图书ID: 001
书名：《Python编程:从入门到实践》
作者：[美]埃里克·马瑟斯(Eric Matthes)
出版社：人民邮电出版社
是否可借：True
借阅次数：4
-----
图书ID: 002
书名：《流畅的Python》
作者：Luciano Ramalho
出版社：人民邮电出版社
```

```
是否可借: True
借阅次数: 0
-----
图书ID: 003
书名: 《Python核心编程》
作者: Wesley J. Chun
出版社: 人民邮电出版社
是否可借: True
借阅次数: 0
-----
图书ID: 005
书名: 《Python数据科学手册》
作者: Jake VanderPlas
出版社: 人民邮电出版社
是否可借: True
借阅次数: 0
-----

【10】保存数据到文件:
数据已保存到library_data.json

【11】从文件加载数据:
已从library_data.json加载数据: 图书4本, 借阅记录0条
```

七、实验总结与心得:

1. 通过完成"简易图书馆资源管理系统"实验,我在以下方面收获颇丰:

1)

技术层面:

熟练掌握了Python核心数据结构(列表、字典)的综合运用

深入理解了面向对象编程思想,能够合理设计类结构

学会了使用函数封装实现代码模块化

掌握了文件操作实现数据持久化

2)

思维层面:

培养了系统化的问题分析能力

提升了算法设计和逻辑思维能力

增强了代码调试和错误处理能力

2. 遇到的问题及解决方法

1)

图书ID重复问题:

现象: 添加图书时未检查ID唯一性

解决: 在 add_book 方法中添加ID存在性检查

2)

借阅状态同步问题:

现象: 借阅记录与图书状态不一致

解决: 使用事务性操作,确保两个操作同时成功

3)

模糊查询大小写敏感:

现象: 查询"python"无法找到"Python"相关书籍

解决: 统一转换为小写进行比较

3. 对 OOP 的理解提升

通过本次实验, 我深刻体会到面向对象编程的优势:

- 1) 封装性: 将图书数据和行为封装在Book类中, 提高代码安全性
- 2) 继承性: 为后续扩展(如电子图书、期刊)奠定基础
- 3) 多态性: 通过统一接口处理不同类型查询, 提高代码灵活性

4. 心得体会

本次实验让我认识到, 良好的系统设计是高质量软件的基础。通过实际动手实现, 我不仅巩固了Python编程技能, 更重要的是培养了"自顶向下、逐步求精"的系统思维方法。在解决实际问题的过程中, 我深刻体会到编程不仅是技术活, 更是需要耐心和细心的手艺活。未来我将继续深入学习, 不断提升自己的编程能力和系统设计水平。

实验诚信说明: 本人(朱明钊)独立完成此实验

AI工具使用说明: 仅在独立完成代码基础的前提下, 请求Github Copilot检查并加以完善。

Book类和Library类皆为本人独立编写。附加功能(数据持久化, 可视化界面)则请求了AI提供代码支持。

我正在完成Python大作业，利用Python搭建一个图书管理系统。请你根据我...

...

1. Book类完善

- `to_dict` 方法中的 `"is_available":self.publisher`, 应为 `"is_available":self.is_available`。
- 建议将 `print_book_info` 方法中的内容简化为只打印自身属性，不要包含示例数据。

2. Library类完善

2.1 方法缩进和结构修正

你的 `modify_book`、`remove_book`、`borrow_book`、`return_book` 方法都嵌套在 `search_book` 内部了，应该和 `search_book` 平级。否则这些方法无法被正常调用。

2.2 逻辑修正

- `add_book` 方法中 `self.find_book_by_id(self,book_id)` 应为 `self.find_book_by_id(book_id)`。
- `search_book` 方法中 `book.title()` 应为 `book.title`。
- `remove_book` 方法中的 `printf` 应为 `print`。
- `book.print_book_info(book)` 应为 `book.print_book_info()`。
- 删除图书时，最后一行 `print(f"未找到图书《{book.title}》")`，此时 `book` 可能未定义，建议改为 `print(f"未找到图书ID{book_id}")`。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。